Recurrent Neural Networks (RNN)

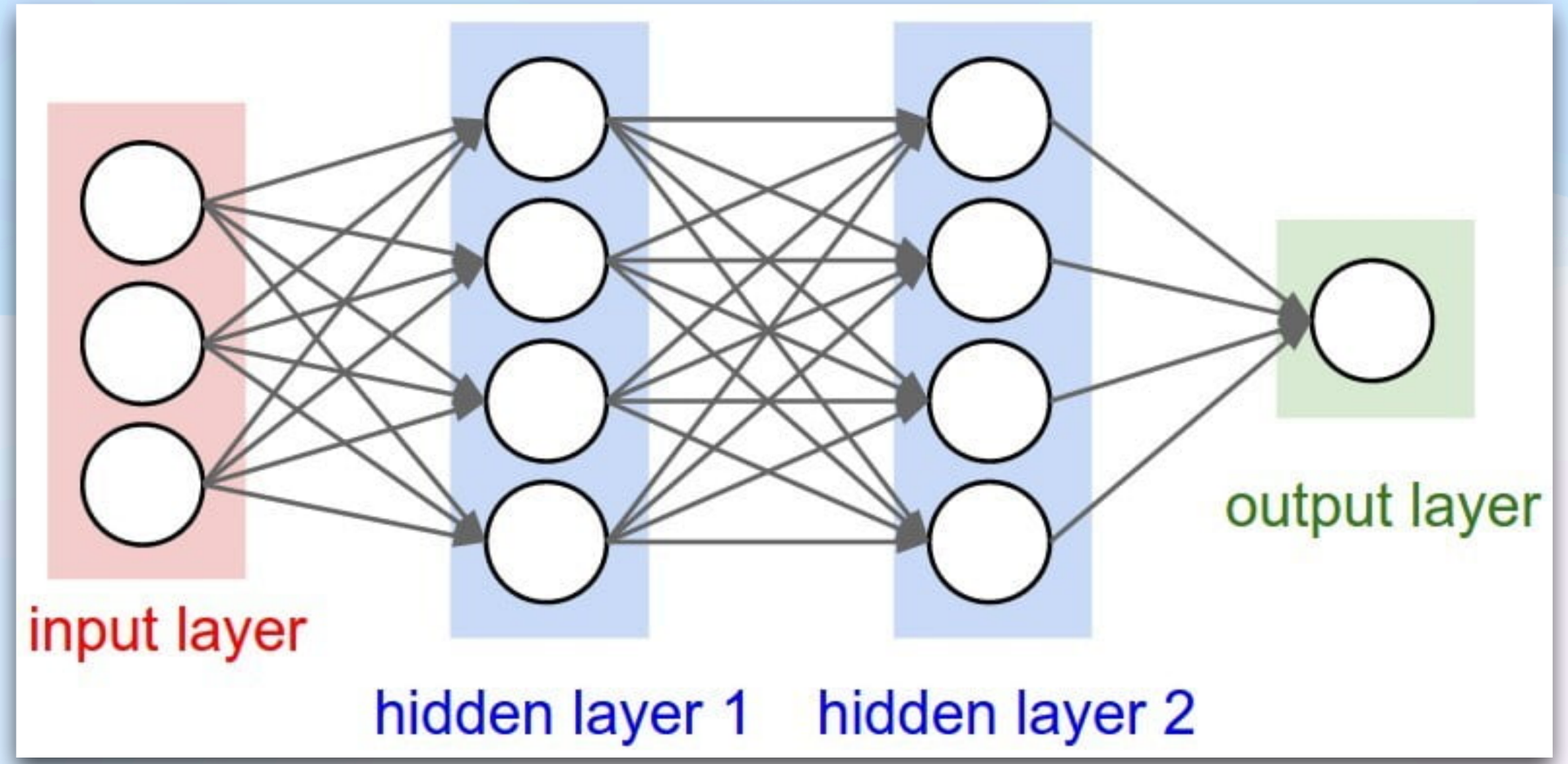
LauzHack Deep Learning Summer Bootcamp

Seyed Parsa Neshaei — July 2025

Several examples and material inspired by the book Deep Learning with Python (François Chollet)

Let’s have a recap!

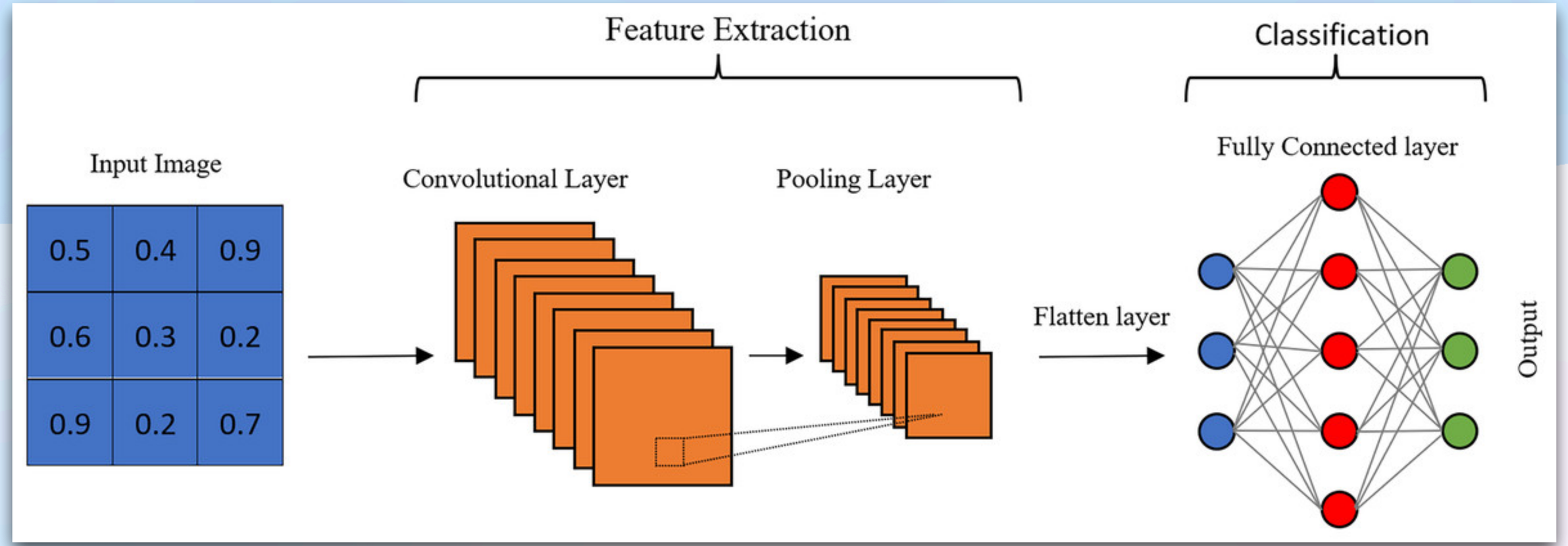
Deep Neural Networks



https://www.bmc.com/blogs/deep-neural-network/

Let’s have a recap!

Convolutional Neural Networks

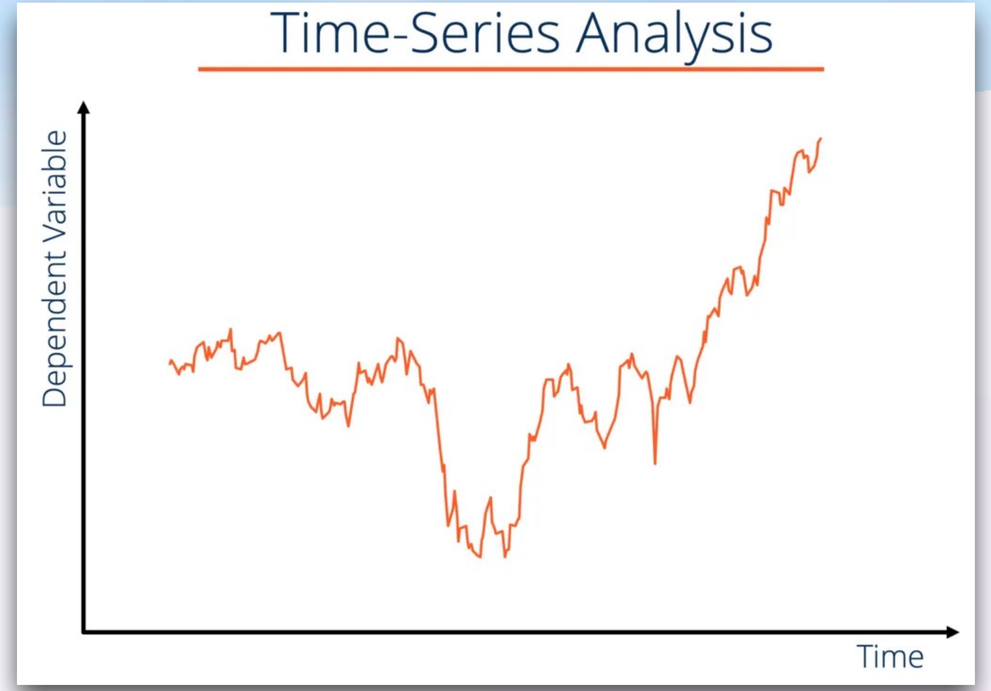


https://peerj.com/articles/cs-1395/

A new task!

Time series

- Data measured at regular intervals, or step-by-step. Examples:
  - Hourly temperature in a city
  - Heights of ocean tides
  - Heartbeats per minute
  - The stock price of a company
  - Answers of students to exam questions
  - A written text (!): *EPFL I like* vs. *I like EPFL*
- Tasks: *classification* or *forecasting*
- Tip: if data semantics change by permutation



https://corporatefinanceinstitute.com/resources/data-science/time-series-data-analysis/

Time Series

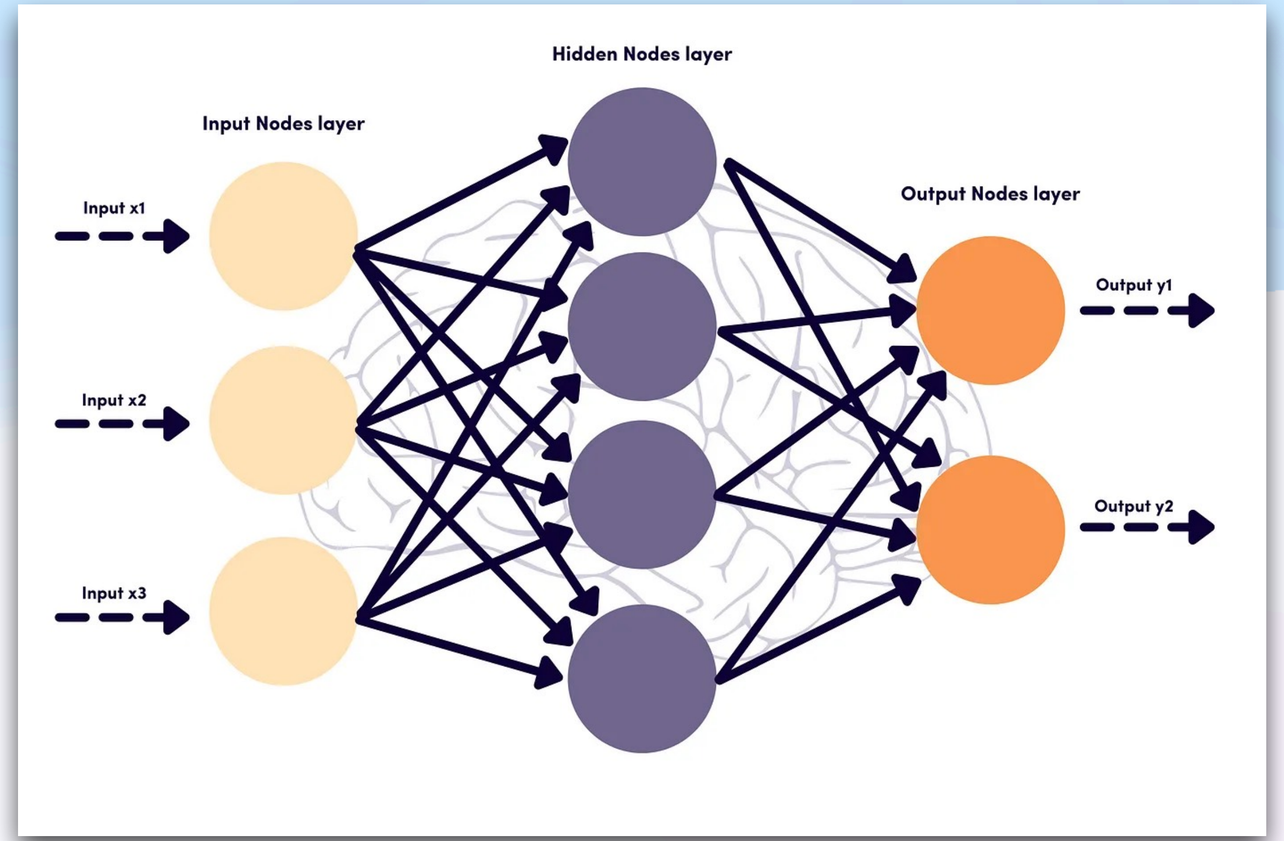
Is classifying/forecasting time series data obvious?

- The model needs to extract trends, rises/falls, cycles, etc.

What is a downside of NNs until now?

No Memory

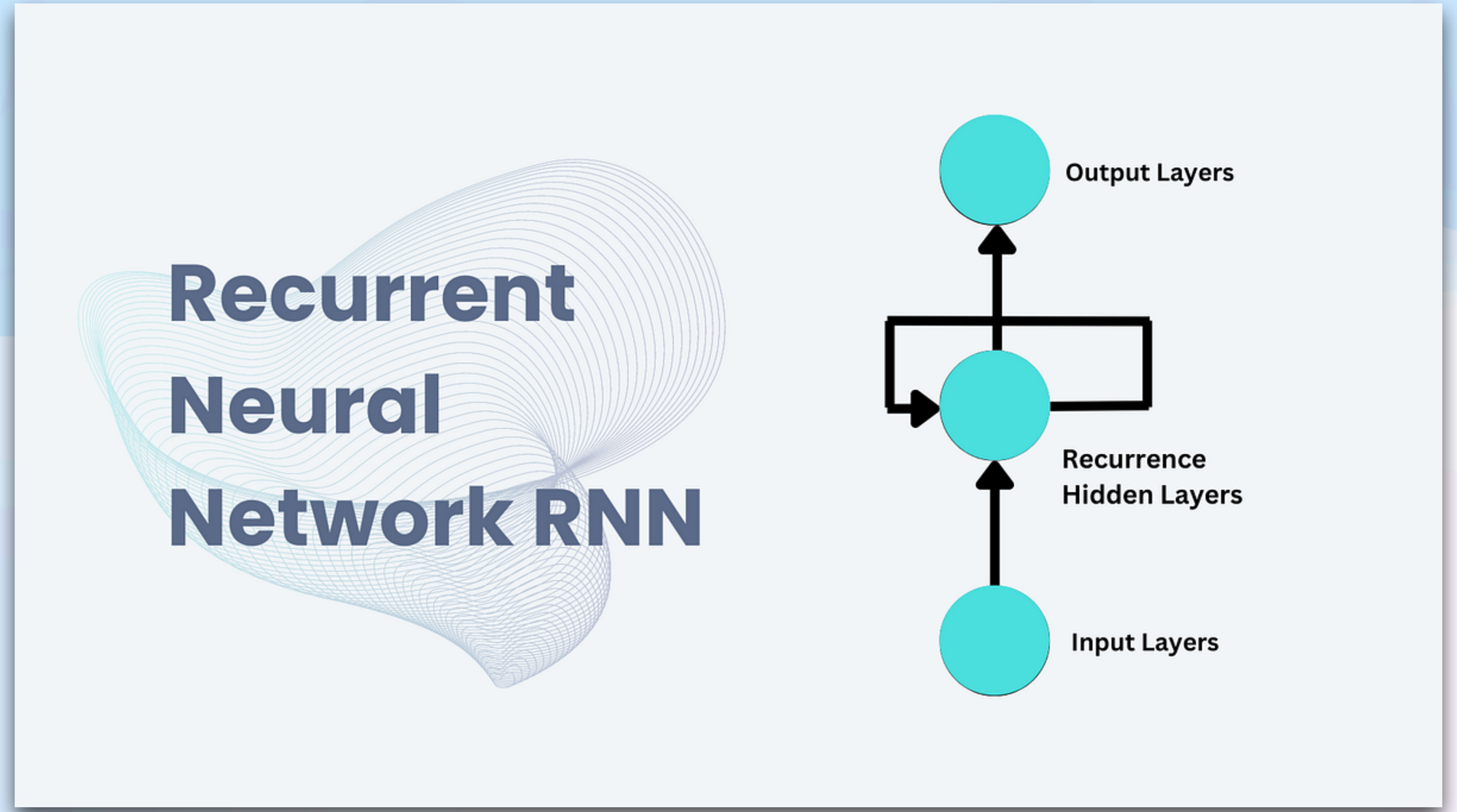
- The processing of each input entry happens independently
- You should *flatten* your data (the whole sequence)
- But time series data come gradually!
- We want to have an *internal state* that changes *over time* (is constantly updated)



https://medium.com/@praveenraj.gowd/understanding-recurrent-neural-networks-rnn-nlp-e2f4cae03a4f

Enter RNNs!

Recurrent Neural Networks



https://medium.com/@praveenraj.gowd/understanding-recurrent-neural-networks-rnn-nlp-e2f4cae03a4f

Enter RNNs!

Recurrent Neural Networks

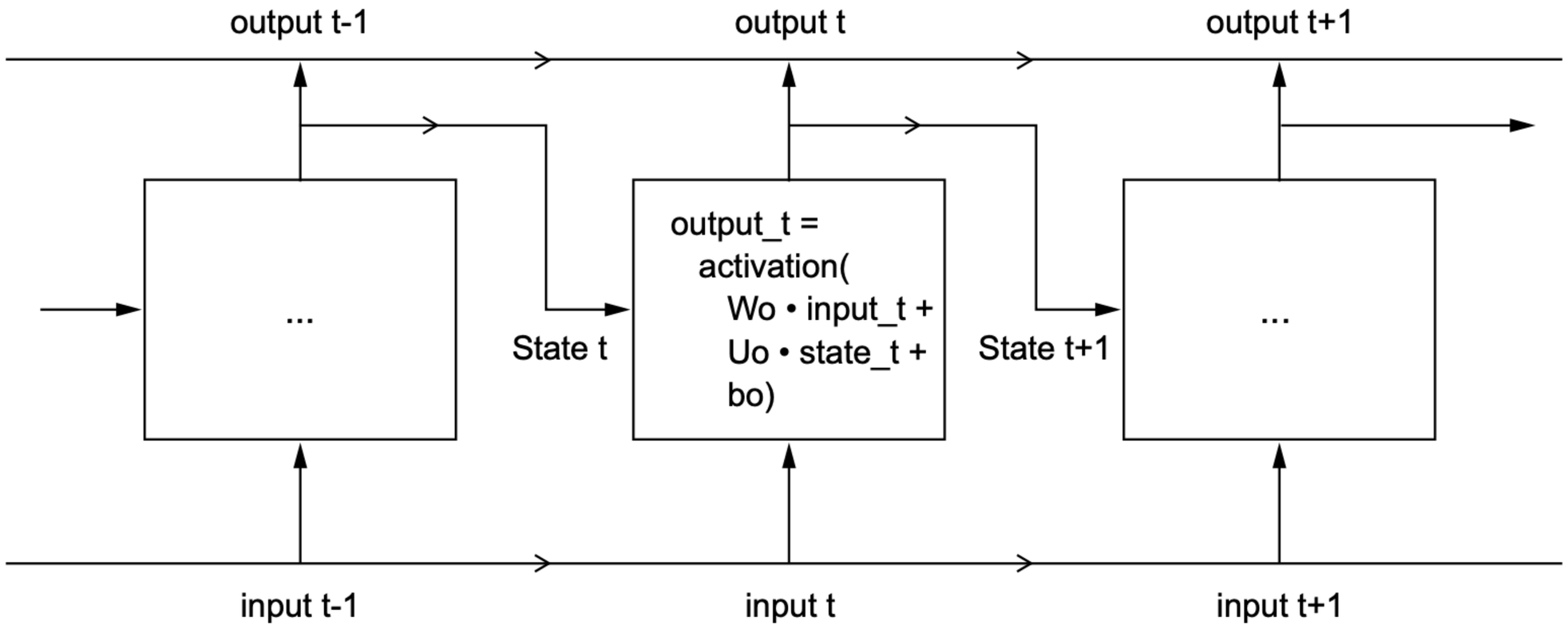


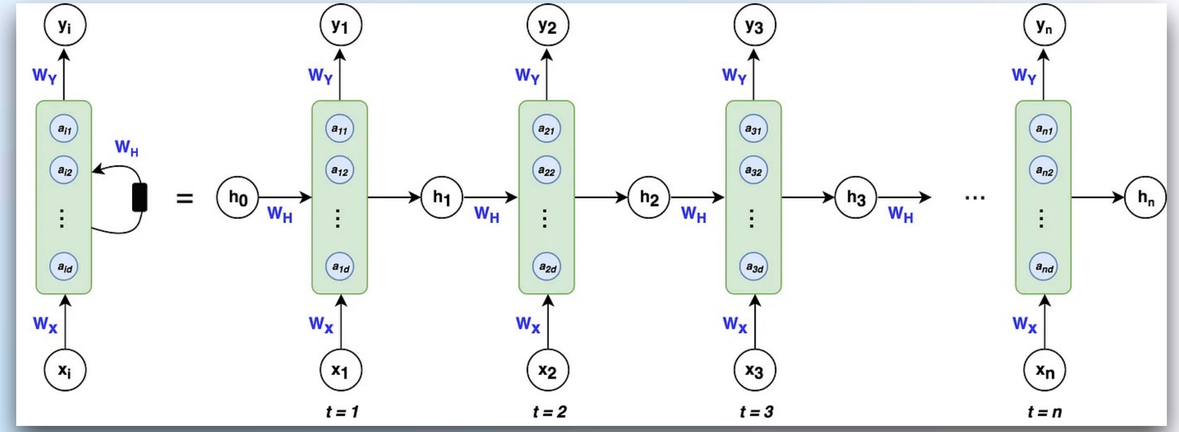
Figure 10.7 A simple RNN, unrolled over time

From the book Deep Learning with Python (François Chollet)

Let’s see it in code!

Recap

- RNNs included a *hidden/memory state* over time, remembering information about sequences
  - Better sequence matching, useful for time series (and textual!) data
  - *Question:* think about how you can make an autocomplete system using it …?



Images from https://medium.com/@praveenraj.gowd/understanding-recurrent-neural-networks-rnn-nlp-e2f4cae03a4f

$$a_t = \boxed{W_H h_{t-1} + W_X X_t}$$ **Hidden Nodes**

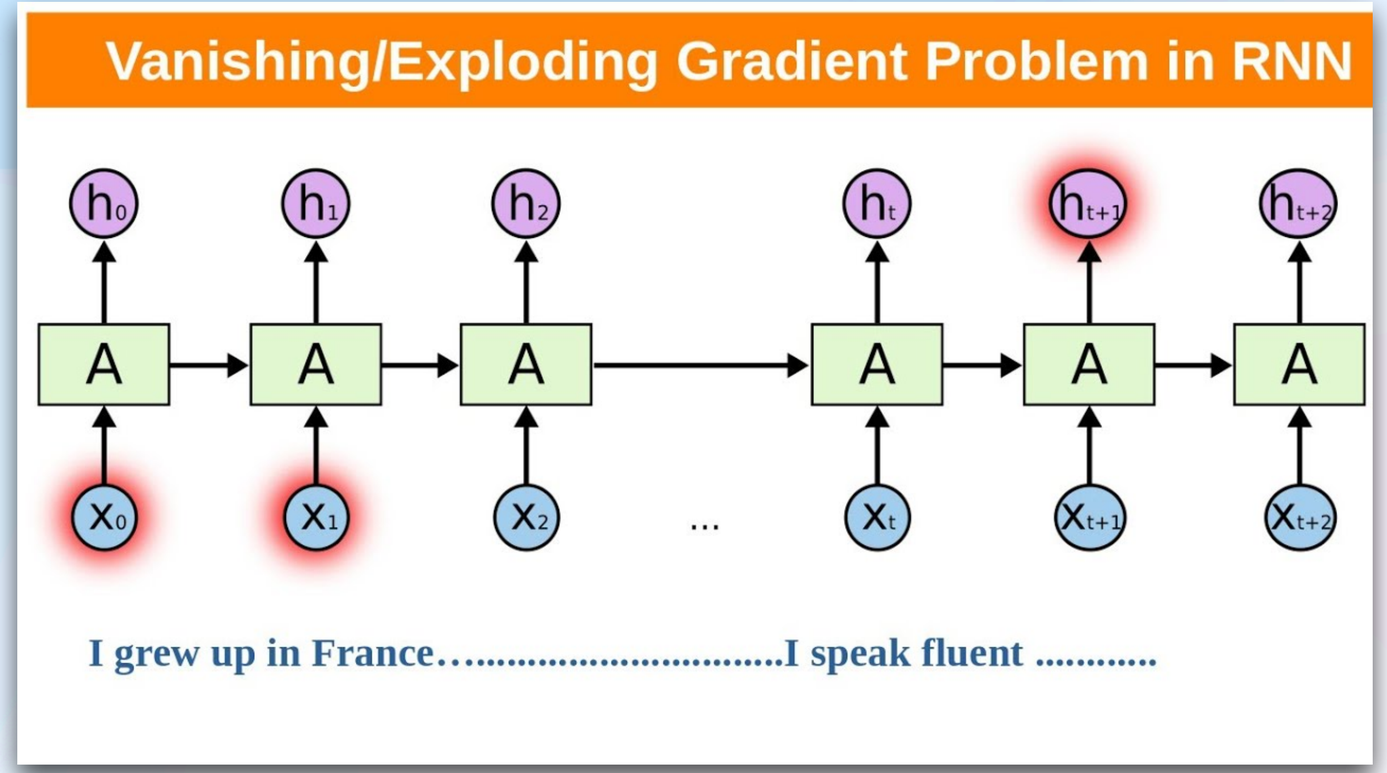
Activation Function

Output From Hidden State → $h_t = \underbrace{\boxed{tanh(\boxed{a_t})}}_{\text{Hidden State}}$

Prediction at time t → $y_t = softmax(W_Y h_t)$

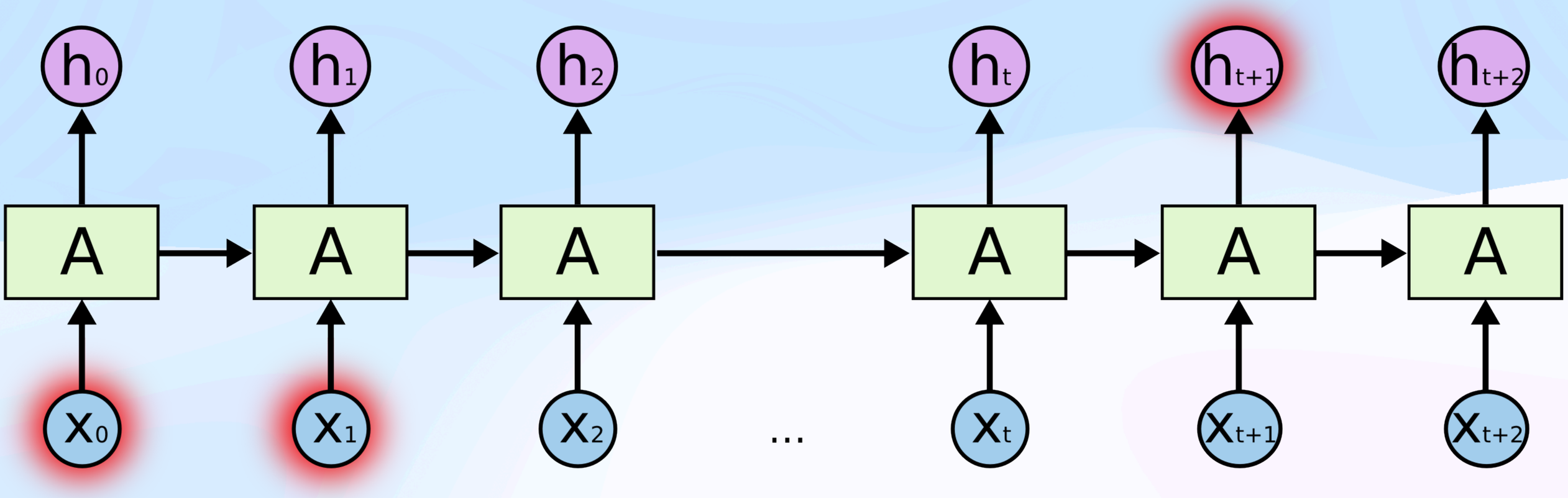
What if the pattern continues?

- Vanishing gradient problem: multiplying chain rule operations < 1
- Makes it difficult for RNNs to predict based on entries occurring at the start of long sequences (almost no weight update) — long term dependency…



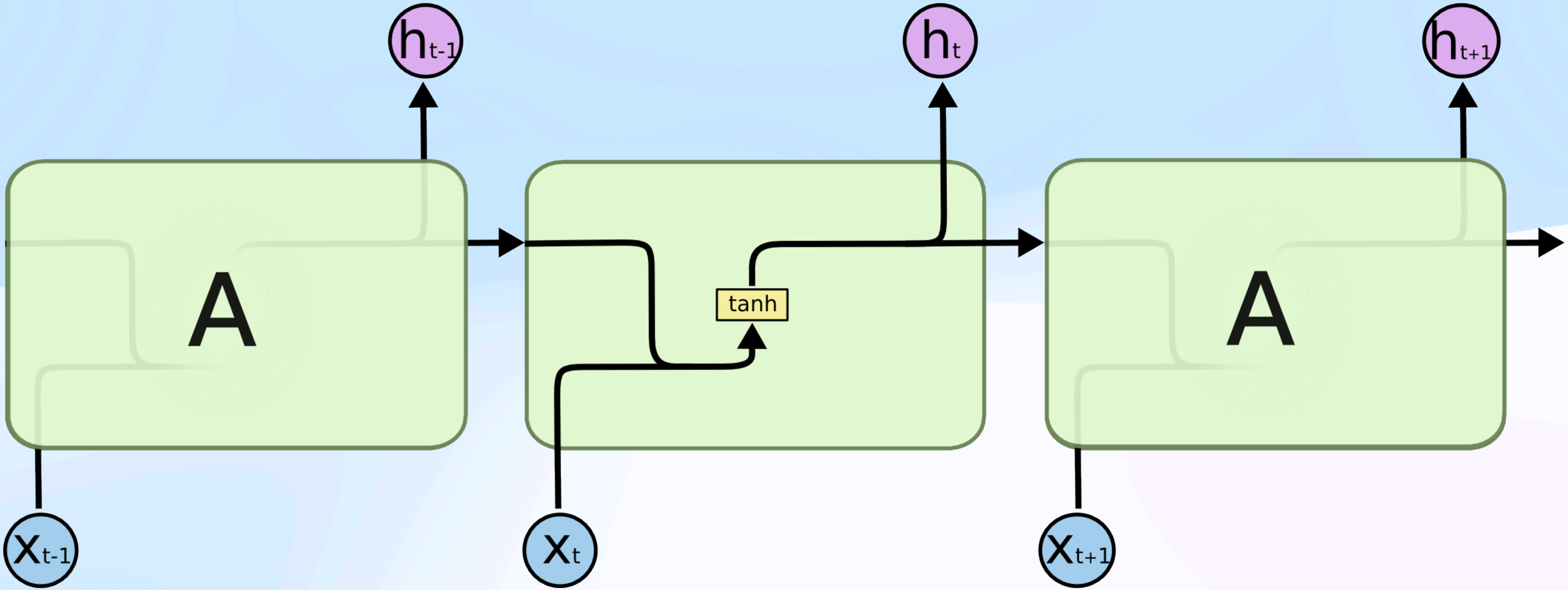
https://www.youtube.com/watch?app=desktop&v=nLPRvGogtrI

So, this was an RNN…



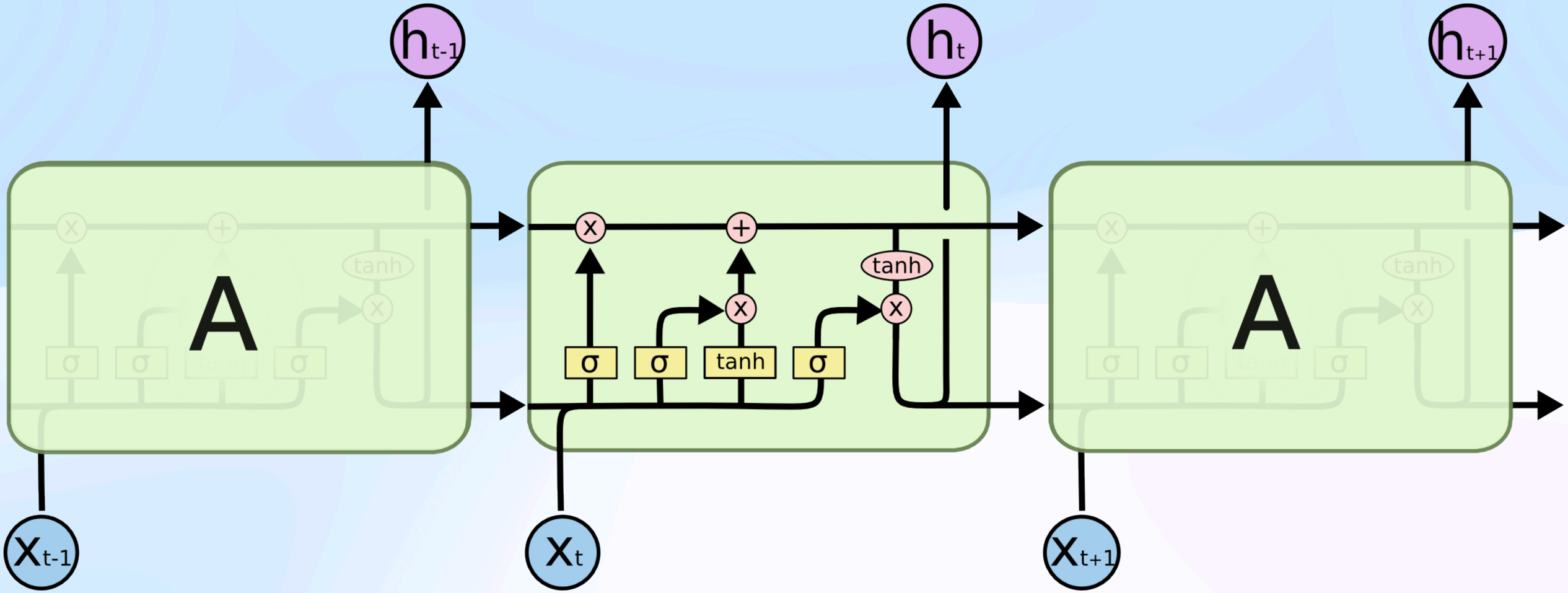
https://colah.github.io/posts/2015-08-Understanding-LSTMs/

So, this was an RNN…



https://colah.github.io/posts/2015-08-Understanding-LSTMs/

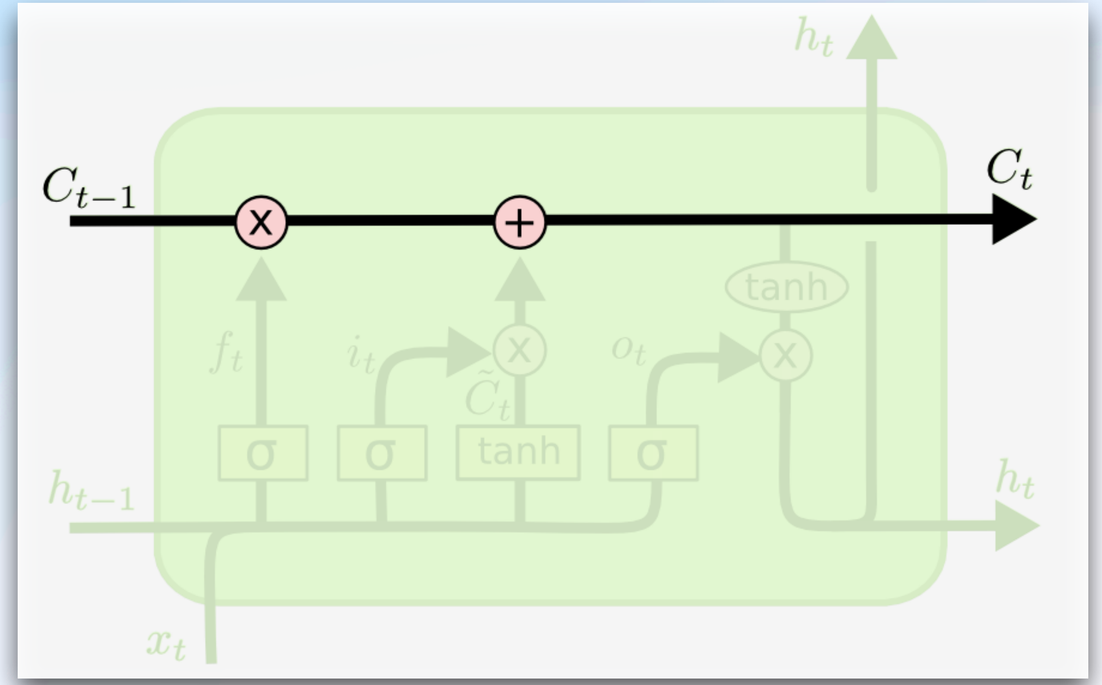
Enter LSTMs!



https://colah.github.io/posts/2015-08-Understanding-LSTMs/

Cell State

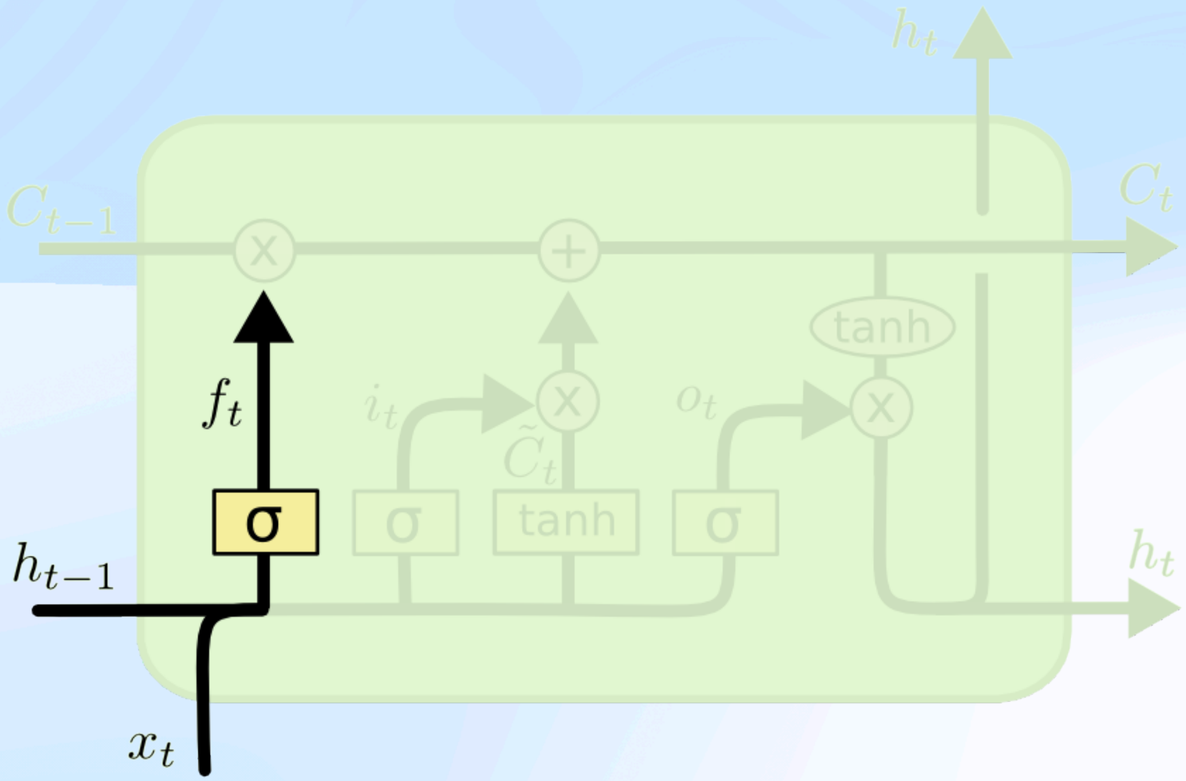
- A line running in the whole chain (info can even flow without change)
- Data can be added/removed to the cell state via several *gates*
  - Output of sigmoid between zero (let nothing go) and one (let everything go)



https://colah.github.io/posts/2015-08-Understanding-LSTMs/

1) Forget Gate

- Example: forgetting information after a new subject in a sentence

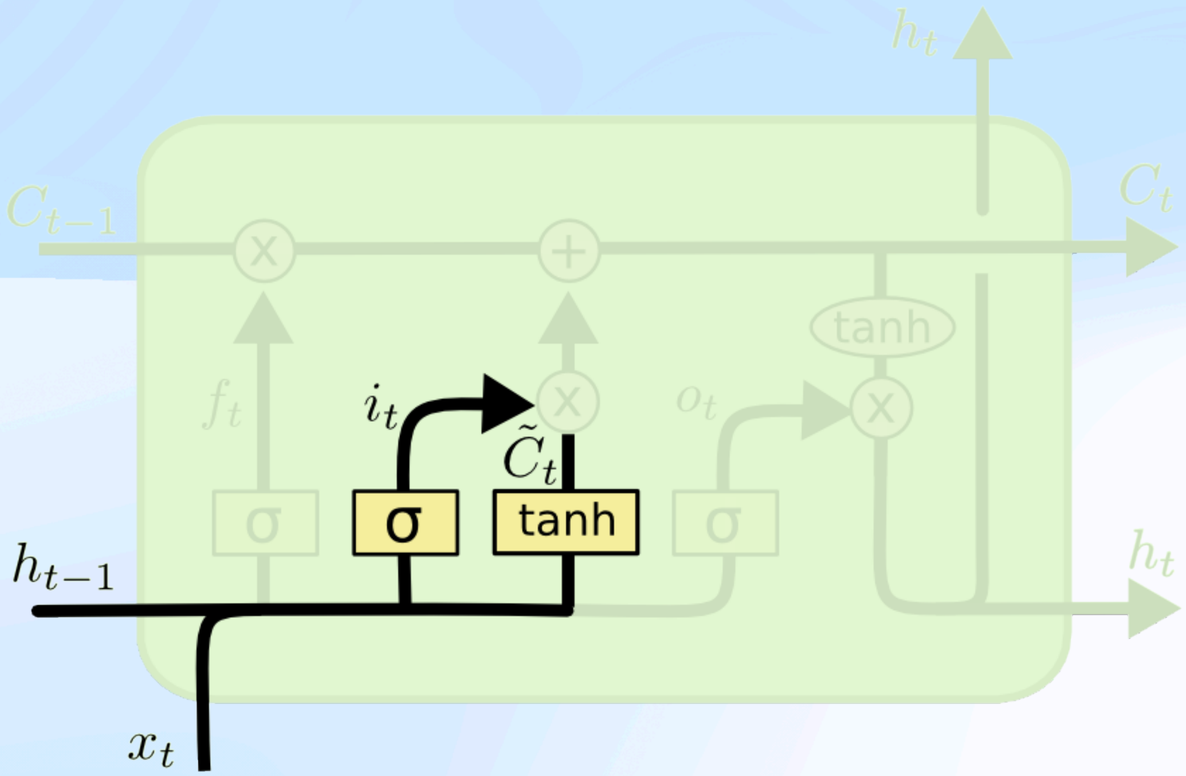


$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

https://colah.github.io/posts/2015-08-Understanding-LSTMs/

2) Input Gate

- What new information are we going to store in the cell state

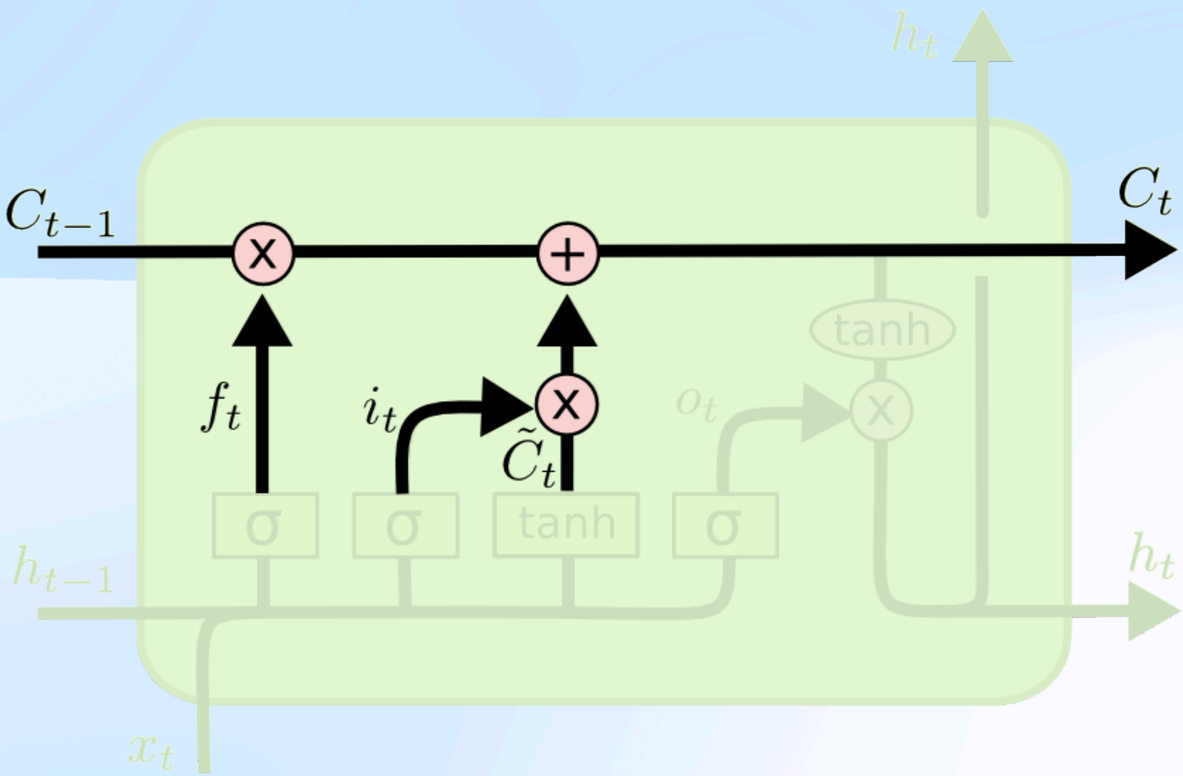


$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \ + \ b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \ + \ b_C)$$

https://colah.github.io/posts/2015-08-Understanding-LSTMs/

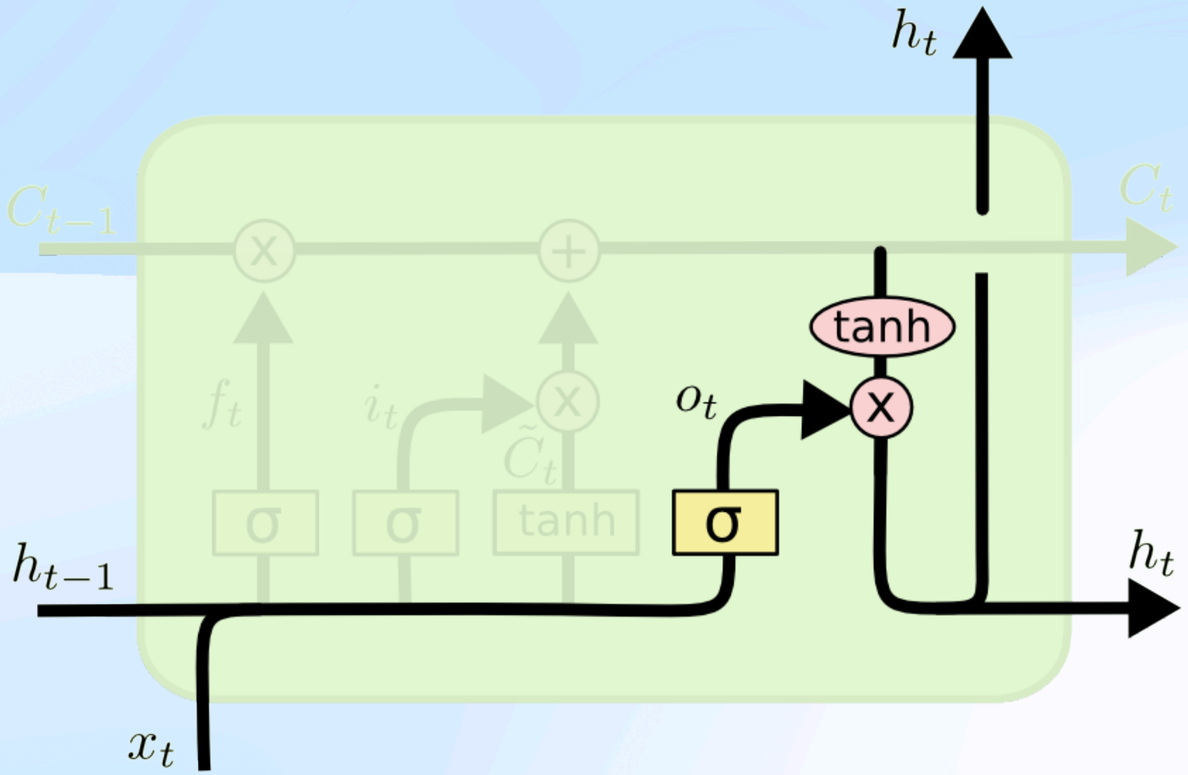
New Cell State



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

https://colah.github.io/posts/2015-08-Understanding-LSTMs/

3) Output Gate



$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

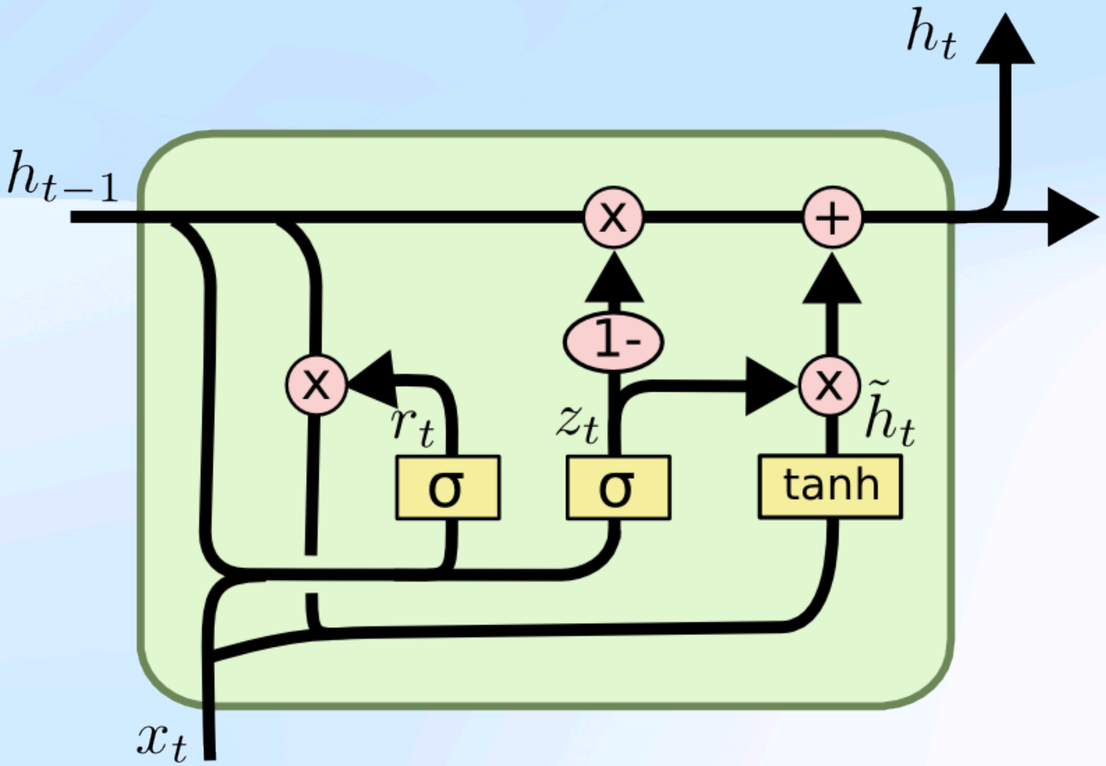
$$h_t = o_t * \tanh\left(C_t\right)$$

https://colah.github.io/posts/2015-08-Understanding-LSTMs/

Let’s see it in code!

A variation: Gated Recurrent Unit (GRU)

- Combines forget and input gates into an *update* gate
- Merges the cell state and hidden state



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

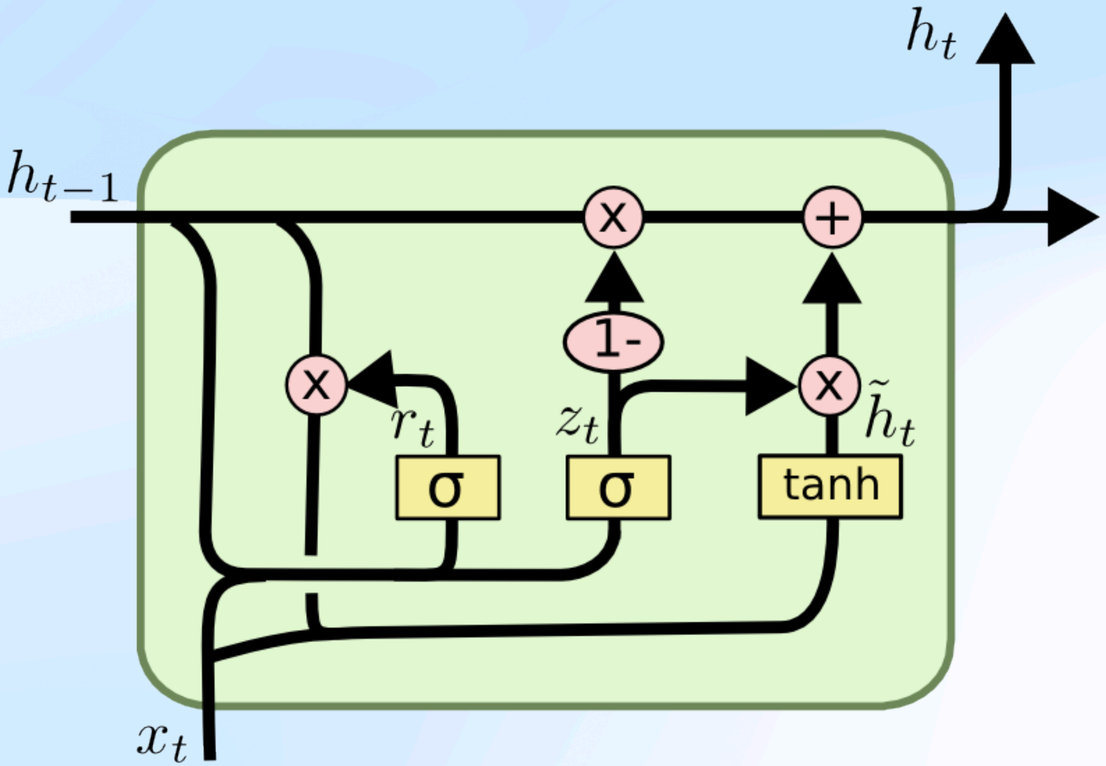
$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

https://colah.github.io/posts/2015-08-Understanding-LSTMs/

Reset Gate in GRU

- Changes the hidden state (short-term memory)



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

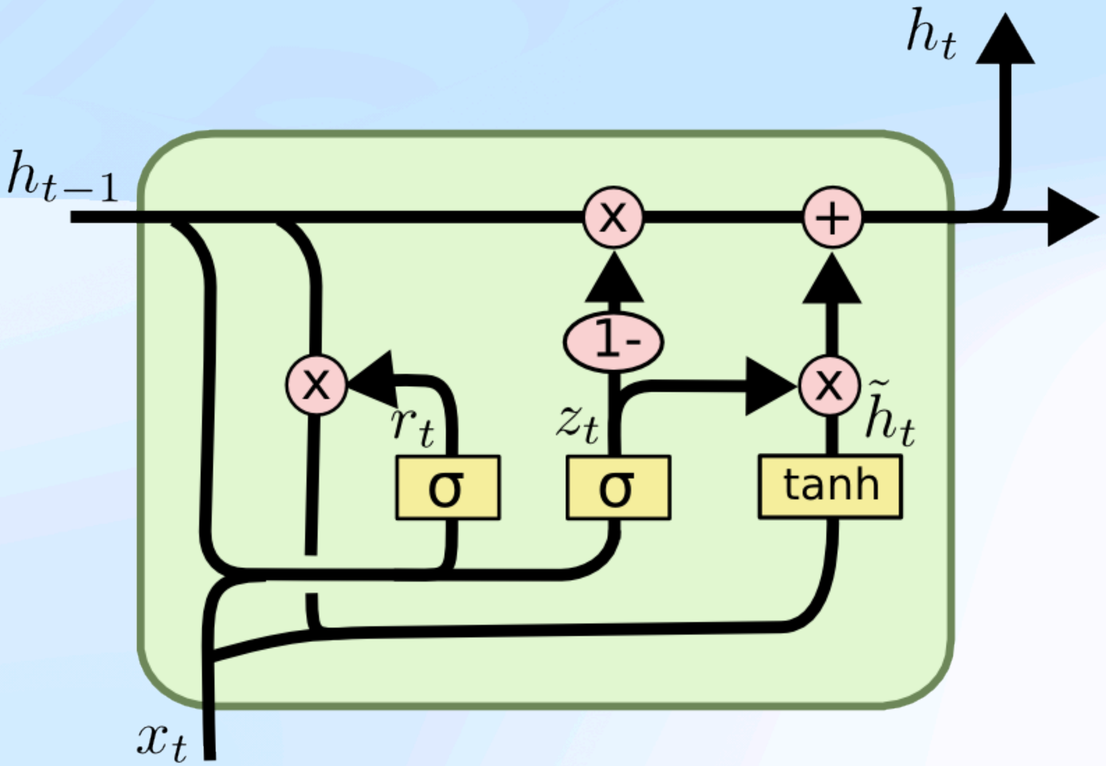
$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

https://colah.github.io/posts/2015-08-Understanding-LSTMs/

Update Gate in GRU

- For long-term memory



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

https://colah.github.io/posts/2015-08-Understanding-LSTMs/

Let’s see it in code!

Recap

- LSTMs and GRUs can help mitigate the vanishing gradient problem
- Also useful for text (come to the NLP lecture during the next week!)
  - Embeddings
- One main concern: limited interpretability
- The next step is attention and transformers -> tomorrow!

A real case study!

Student Knowledge Tracing

- Students interact with a learning system over time and answer questions
  - Imagine assignments in a massive online open course
- Students answer questions of certain topics right or wrong
- Can we predict their answers to the next questions?
  - Why is this useful?

Student

Question	1	2	3	4	5	6	7	8	9	10	11
Skill ID	**2**	3	1	1	**2**	1	3	3	**2**	1	**2**
Student Answer	✔	✘	✔	✔	✘	✘	✔	✘	✔	✔	**?**

Neshaei, Seyed Parsa, et al. "Towards modeling learner performance with large language models." Educational Data Mining Conference Posters (2024).

A real case study!

Student Knowledge Tracing

- Researchers (e.g., Käser et al. (2017)) have explored Bayesian and Markov models to predict the *mastery* of each skill among students
- Piech et al. (2015) achieved state-of-the-art-at-the-time results with LSTMs

	Overview			*AUC*			
Dataset	Students	Exercise Tags	Answers	Marginal	BKT	BKT*	DKT
Simulated-5	4,000	50	200 K	?	0.54	-	0.75
Khan Math	47,495	69	1,435 K	0.63	0.68	-	0.85
Assistments	15,931	124	526 K	0.62	0.67	0.69	0.86

Table 1: AUC results for all datasets tested. BKT is the standard BKT. BKT is the best reported result from the literature for Assistments. DKT is the result of using LSTM Deep Knowledge Tracing.*

Piech, Chris, et al. "Deep knowledge tracing." Advances in neural information processing systems 28 (2015).

Let’s see it in code!

Some more details on "text"

Natural Language Processing

- Any task that works with textual data
- Imagine a model saying if a certain sentence is happy or sad
- Three components:

  1) Represent words in a computer-understandable way: **embeddings**

  2) The model that gets the words as input

  3) Mapping between model outputs and predictions (e.g., the last layer)

- You already know 2 and 3…

Tokenization

- Each sentence contains several words
- Each “token” is a meaningful unit of characters -> discretization!
- “I am attending the summer bootcamp!”
  - I + am + attending + the + summer + bootcamp + !
- *Is this the only way to tokenize the sentence above?*

Why not each word as a token?

- “I am attending the summer bootcamp!”
  - I + am + attending + the + summer + bootcamp + !
- How do you tokenize “check-in” or “six-years-old” or “isn’t”?
- “attend” and “attending” are then totally different tokens, so why not be close?
  - The number of words increases a lot for each variation -> computation
- Some languages do not use space as English does: Korean, Chinese, etc.
- Example alternative:
  - I + am + attend + ing + the + summer + boot + camp + !
- Specific tokens might be needed, e.g., <UNK>

Vocabulary

- Set of all possible tokens
- Any token not in the vocabulary $|V|$ -> <UNK>
- We should represent each word of $|V|$ as a *vector*
- One early solution: one-hot vectors of size $|V|$
  - I -> [1, 0, 0, 0, …]
  - am -> [0, 1, 0, 0, …]
  - attend -> [0, 0, 1, 0, …]
- Downside?

Embeddings

- We want word vectors to contain semantic information
- One-hot does not provide it; how can we find the similarity between these vectors?
  - like -> [0 0 …… 1 …… 0]
  - love -> [0 0 …… 1 …… 0]
  - parameter -> [0 0 …… 1 …… 0]
- Normal time series has semantic information: numbers can be huge, small, or have patterns
  - So, in time series for NLP, the inputs (word vectors) should have meanings
- We should use vectors with real numbers as entries, but how to find them?

Embeddings: learn from the model

- One possible solution: add feedforward networks, converting one-hot to real-valued vectors
- Is it enough?
- Usually labeled training data is rare, not enough to capture enough dependencies between words
- Idea: use vast amounts of unlabeled text (e.g., on the internet) to find the embeddings, then use those embeddings in our models trained using labeled data
  - Fictional example: use the whole Wikipedia (billions of words) to find embeddings, then use them to train a model on 1000 sentences to classify sad/happy

Self-supervised Learning of Embeddings

- Find patterns by looking at large bodies of text and words that come next to each other
  - *Imagine a model looking at the whole internet; it should learn that Bern, Zurich, Lausanne, etc. come with the word Switzerland regularly, so they are Swiss cities*
- One approach: a model to *predict* a word in a gap, from the surrounding
  - I am ……. the summer bootcamp!
    - attending, enjoying, …

Let’s see in the code!

Thank you!